TrustChain-AI

Architecture & Diagrams Technical Reference Collection

Project: A Decentralized Trust Intelligence Framework for Secure Autonomous AI Agent Collaboration

Department: Computer Engineering, MPSTME, SVKM's NMIMS University, Mumbai

Document Purpose: Official diagrammatic specification for academic and technical examination

---

Index of Included Diagrams:

#	Diagram Title	Category
1	Overall System Architecture	High-Level Architecture
2	Five-Layer Layered Architecture	Architectural Design
3	System Context Diagram (C4 Level 1)	Context & Boundaries
4	DID:key Generation & Verification Flowchart	Identity & Cryptography
5	Verifiable Credential Issuance & Verification Sequence	Credential Lifecycle
6	Behavioral Trust Evaluation Flowchart	Trust Mathematics
7	Trust State Transition Finite State Machine	State Machine
8	Adaptive Authorization Flow	Security & Policy
9	Agent Discovery & Trust-Weighted Candidate Matching	Multi-Agent Coordination
10	Sequential Multi-Agent Workflow Execution	Workflow Orchestration
11	Database Entity-Relationship Diagram (14 Relational Tables)	Database Architecture
12	Security Architecture & Defense-in-Depth	Cybersecurity & Governance

Figure 1: Overall System Architecture

Category: High-Level Architecture

Technical Description: Illustrates the end-to-end integration across the React 19 Frontend SPA, FastAPI ASGI Backend, Trust Intelligence Core (TrustEngine, AdaptivePolicyEnforcer, LocalAgentConnector), and the 14-table local SQLite Verifiable Data Registry.

Mermaid Diagram Specification:

```
flowchart TB
    subgraph UI["Frontend — React 19 / Vite 8"]
        A1[DashboardHome]
        A2[AgentExplorer]
        A3[TrustAnalytics]
        A4[CollaborationCenter]
        A5[WorkflowCenter]
        A6[IdentityCredentials]
        A7[AuditTrail]
        A8[SystemHealth]
    end

    subgraph API["Backend — FastAPI 0.141 / Uvicorn"]
        B1[Auth Router /auth]
        B2[Registry Router /registry]
        B3[VDR Router /vdr]
        B4[Health Endpoint /health]
    end

    subgraph CORE["Trust Intelligence Core"]
        C1[TrustEngine — EMA Math]
        C2[AdaptivePolicyEnforcer]
        C3[LocalAgentConnector]
        C4[DIDKey Generator]
        C5[VerifiableCredential]
    end

    subgraph DB["Local SQLite VDR — vdr.db (WAL Mode)"]
        D1[(agent_trust_states)]
        D2[(telemetry_logs)]
        D3[(trust_audit_trails)]
        D4[(agents)]
        D5[(stored_vcs)]
        D6[(workflows & nodes)]
        D7[(collaboration_sessions)]
        D8[(users & refresh_tokens)]
    end

    UI -->|Axios REST / JWT Bearer| API
    API --> CORE
    CORE --> DB
```

Figure 1 — Overall System Architecture (TrustChain-AI Specification)

Figure 2: Five-Layer Layered Architecture

Category: Architectural Design

Technical Description: Details the strict separation of concerns from presentation down to persistence, ensuring modularity, testability, and standard-aligned interoperability.

Mermaid Diagram Specification:

```
flowchart TB
    L1["Layer 1: Presentation Layer
React 19.2 · Vite 8.2 · Recharts · Tailwind CSS · Dark/Light Mode"]
    L2["Layer 2: API Gateway Layer
FastAPI 0.141 · Uvicorn ASGI · JWT Authentication · RBAC · Pydantic v2"]
    L3["Layer 3: Trust Intelligence Layer
TrustEngine (EMA Scoring) · AdaptivePolicyEnforcer · LocalAgentConnector"]
    L4["Layer 4: Identity & Credential Layer
DID:key (Ed25519 Multibase) · VerifiableCredential (Ed25519Signature2020)"]
    L5["Layer 5: Persistence Layer
SQLite VDR (WAL Mode) · SQLAlchemy 2.0 ORM · 14 Relational Tables"]

    L1 --> L2 --> L3 --> L4 --> L5
```

Figure 2 — Five-Layer Layered Architecture (TrustChain-AI Specification)

Figure 3: System Context Diagram (C4 Level 1)

Category: Context & Boundaries

Technical Description: Depicts external human actors (Admin, Operator, Viewer) and Autonomous AI Agents interfacing with the TrustChain-AI boundary over HTTPS and REST interfaces.

Mermaid Diagram Specification:

```
flowchart TB
    subgraph Actors["External Actors"]
        A1[System Administrator — Full RBAC]
        A2[Operations Engineer — Agent Operator]
        A3[Security Auditor — Read-Only Viewer]
        A4[Autonomous AI Agents — Workflows & Tasks]
    end

    subgraph Platform["TrustChain-AI Platform Boundary"]
        FE[React 19 Web Dashboard]
        BE[FastAPI REST Application (:8050)]
        TE[Trust & Policy Evaluation Core]
        DB[(SQLite Verifiable Data Registry)]
    end

    A1 -->|Browser / HTTPS| FE
    A2 -->|Browser / HTTPS| FE
    A3 -->|Browser / HTTPS| FE
    A4 -->|REST API / Bearer Token| BE
    FE -->|JSON REST Requests| BE
    BE --> TE
    TE --> DB
```

Figure 3 — System Context Diagram (C4 Level 1) (TrustChain-AI Specification)

Figure 4: DID:key Generation & Verification Flowchart

Category: Identity & Cryptography

Technical Description: Step-by-step cryptographic sequence for generating W3C-conformant did:key identifiers using PyNaCl Ed25519 keypairs and multibase base58btc encoding.

Mermaid Diagram Specification:

```
flowchart TD
    Start([Generate Key Request]) --> K[PyNaCl: Generate Ed25519 Keypair]
    K --> Raw[Extract 32-Byte Public Key]
    Raw --> Pre[Prepend Multicodec Header: 0xED 0x01]
    Pre --> B58[Base58BTC Encode Prefixed Bytes]
    B58 --> Z[Prefix with 'z' -> Multibase String]
    Z --> DID[Format URI: did:key:z{multibase}]
    DID --> Doc[Assemble W3C DID Document:
- VerificationMethod: Ed25519VerificationKey2020
- Relationships: authentication, assertionMethod]
    Doc --> Out([Return DID + Document + Private Key])
```

Figure 4 — DID:key Generation & Verification Flowchart (TrustChain-AI Specification)

Figure 5: Verifiable Credential Issuance & Verification Sequence

Category: Credential Lifecycle

Technical Description: UML sequence diagram illustrating VC creation, URDNA2015 canonicalization via PyLD, Ed25519 signing via PyNaCl, and cryptographic verification against issuer public keys.

Mermaid Diagram Specification:

```
sequenceDiagram
    participant Issuer as Issuer Agent / Admin
    participant VDR as VDR Service (/vdr)
    participant PyLD as PyLD (URDNA2015)
    participant PyNaCl as PyNaCl (Ed25519)
    participant DB as SQLite stored_vcs

    Issuer->>VDR: POST /vdr/credentials (claims, subject_did)
    VDR->>PyLD: Canonicalize JSON-LD structure (URDNA2015)
    PyLD-->>VDR: Deterministic byte payload
    VDR->>PyNaCl: Sign canonical payload with Issuer Private Key
    PyNaCl-->>VDR: 64-byte Ed25519 digital signature
    VDR->>VDR: Assemble Ed25519Signature2020 proof block
    VDR->>DB: Persist complete VC in stored_vcs
    VDR-->>Issuer: Return Verified Credential JSON-LD
```

Figure 5 — Verifiable Credential Issuance & Verification Sequence (TrustChain-AI Specification)

Figure 6: Behavioral Trust Evaluation Flowchart

Category: Trust Mathematics

Technical Description: Flowchart representing the ingestion of telemetry events, calculation of multi-factor performance score P_i(t), temporal smoothing via EMA T_i(t), and state transition evaluation.

Mermaid Diagram Specification:

```
flowchart TD
    A[Telemetry Submission: success, compliance, latency] --> B[Log into telemetry_logs]
    B --> C[Query Last n=10 Events for Agent DID]
    C --> D["Calculate Success Rate S_i = successes / n"]
    D --> E["Calculate Compliance Rate C_i = compliances / n"]
    E --> F["Calculate Normalised Latency L_i = min(avg_lat/2000, 1.0)"]
    F --> G["Compute Performance: P_i(t) = 0.6*S_i + 0.3*C_i + 0.1*(1-L_i)"]
    G --> H["Load Prior Temporal Score T_i(t-1) (default 1.0)"]
    H --> I["Compute EMA: T_i(t) = 0.7*P_i(t) + 0.3*T_i(t-1)"]
    I --> J{T_i(t) Threshold Check}
    J -->|T >= 0.8| K[State: TRUSTED]
    J -->|0.5 <= T < 0.8| L[State: SUSPICIOUS]
    J -->|T < 0.5| M[State: QUARANTINED]
    K --> N[Update agent_trust_states]
    L --> N
    M --> N
    N --> O[Write audit trail record into trust_audit_trails]
```

Figure 6 — Behavioral Trust Evaluation Flowchart (TrustChain-AI Specification)

Figure 7: Trust State Transition Finite State Machine

Category: State Machine

Technical Description: Finite State Automaton showing bidirectional dynamic transitions between TRUSTED, SUSPICIOUS, and QUARANTINED states, and the one-way terminal transition to REVOKED.

Mermaid Diagram Specification:

```
stateDiagram-v2
    [*] --> TRUSTED: Agent Registered (T_0 = 1.0)
    TRUSTED --> SUSPICIOUS: T drops below 0.8 (Degradation)
    TRUSTED --> QUARANTINED: T drops below 0.5 (Severe Failure)
    SUSPICIOUS --> TRUSTED: T recovers >= 0.8 (Healthy Telemetry)
    SUSPICIOUS --> QUARANTINED: T drops below 0.5 (Continued Failure)
    QUARANTINED --> SUSPICIOUS: T recovers >= 0.5 (Telemetry Recovery)
    TRUSTED --> REVOKED: Administrative Revocation
    SUSPICIOUS --> REVOKED: Administrative Revocation
    QUARANTINED --> REVOKED: Administrative Revocation
    REVOKED --> [*]: Terminal State (No Recovery)
```

Figure 7 — Trust State Transition Finite State Machine (TrustChain-AI Specification)

Figure 8: Adaptive Authorization Flow

Category: Security & Policy

Technical Description: Decision tree executed by AdaptivePolicyEnforcer for gating incoming operations based on the requesting agent's live trust classification.

Mermaid Diagram Specification:

```
flowchart TD
    Req[Incoming Agent Request: DID + Operation] --> Query[Fetch Live Trust State from V
DR]
    Query --> Check{Trust State?}
    Check -->|REVOKED| DenyRev[HTTP 403 Forbidden: Permanently Revoked]
    Check -->|QUARANTINED| DenyQ[HTTP 403 Forbidden: Quarantined Agent]
    Check -->|TRUSTED| Allow[HTTP 200: Authorize Operation]
    Check -->|SUSPICIOUS| OpCheck{Operation Category?}
    OpCheck -->|PRIVILEGED| DenyPriv[HTTP 403 Forbidden: Privileged Op Denied]
    OpCheck -->|STANDARD| AllowAudit[HTTP 200: Authorize + Log Security Audit Event]
```

Figure 8 — Adaptive Authorization Flow (TrustChain-AI Specification)

Figure 9: Agent Discovery & Trust-Weighted Candidate Matching

Category: Multi-Agent Coordination

Technical Description: Algorithmic pipeline for filtering available agents by requested capability and computing ranked composite scores based on trust, reputation, success rate, and latency.

Mermaid Diagram Specification:

```
flowchart TD
    Req[Task Request: Required Capability] --> FilterCap[Filter agents table by capabili
ty tag]
    FilterCap --> FilterState[Exclude agents in QUARANTINED or REVOKED states]
    FilterState --> ScoreLoop[For each qualifying agent:]
    ScoreLoop --> MatchMath["Compute match_score =
0.4*T_i + 0.3*(reputation/5.0) + 0.2*success_rate + 0.1*(1 - norm_latency)"]
    MatchMath --> Rank[Sort candidates descending by match_score]
    Rank --> Result[Return ordered list of ranked agent endpoints]
```

Figure 9 — Agent Discovery & Trust-Weighted Candidate Matching (TrustChain-AI Specification)

Figure 10: Sequential Multi-Agent Workflow Execution

Category: Workflow Orchestration

Technical Description: End-to-end execution of a 3-stage sequential workflow pipeline (Research -> Analysis -> Verification) showing data chaining and per-step trust validation.

Mermaid Diagram Specification:

```
sequenceDiagram
    participant Op as Human Operator / API Client
    participant WF as Workflow Orchestrator
    participant Disc as Discovery & Trust Engine
    participant Ag1 as Node 1 (Research Agent)
    participant Ag2 as Node 2 (Analysis Agent)
    participant Ag3 as Node 3 (Verification Agent)

    Op->>WF: POST /registry/workflows/{id}/execute
    WF->>Disc: Verify Node 1 Agent Trust State
    Disc-->>WF: Trust OK (TRUSTED)
    WF->>Ag1: Execute Task (Initial Input Data)
    Ag1-->>WF: Output 1 (Research Dossier)
    WF->>Disc: Verify Node 2 Agent Trust State
    Disc-->>WF: Trust OK (TRUSTED)
    WF->>Ag2: Execute Task (Input = Output 1)
    Ag2-->>WF: Output 2 (Structured Analysis)
    WF->>Disc: Verify Node 3 Agent Trust State
    Disc-->>WF: Trust OK (TRUSTED)
    WF->>Ag3: Execute Task (Input = Output 2)
    Ag3-->>WF: Output 3 (Verification Verdict)
    WF->>Op: Return Workflow Result (Status: completed)
```

Figure 10 — Sequential Multi-Agent Workflow Execution (TrustChain-AI Specification)

Figure 11: Database Entity-Relationship Diagram (14 Relational Tables)

Category: Database Architecture

Technical Description: Entity relationship overview showing primary keys, foreign keys, and relational cardinality across the 14 SQLite VDR tables.

Mermaid Diagram Specification:

```
erDiagram
    agents ||--o{ agent_trust_states : tracks
    agents ||--o{ telemetry_logs : records
    agents ||--o{ trust_audit_trails : logs
    agents ||--o{ agent_ratings : receives
    agents ||--o{ workflow_nodes : assigned
    workflows ||--|{ workflow_nodes : contains
    collaboration_sessions ||--|{ collaboration_events : logs
    users ||--o{ refresh_tokens : owns
    users ||--o{ security_audit_events : generates
```

Figure 11 — Database Entity-Relationship Diagram (14 Relational Tables) (TrustChain-AI Specification)

Figure 12: Security Architecture & Defense-in-Depth

Category: Cybersecurity & Governance

Technical Description: Holistic view of security enforcement layers including Argon2id hashing, JWT access/refresh rotation, RBAC, W3C cryptographic signatures, and immutable audit logs.

Mermaid Diagram Specification:

```
flowchart LR
    subgraph AuthN["Authentication Layer"]
        A1[Argon2id Password Hashes]
        A2[JWT Access Token 30m]
        A3[JWT Refresh Rotation 7d]
        A4[5-Strike Account Lockout]
    end

    subgraph AuthZ["Authorization Layer"]
        B1[RBAC: ADMIN / OPERATOR / VIEWER]
        B2[Adaptive Policy: PRIVILEGED vs STANDARD]
        B3[Trust Gating: SUSPICIOUS / QUARANTINED]
    end

    subgraph Crypto["Cryptographic Primitives"]
        C1[Ed25519 Keypairs via PyNaCl]
        C2[W3C did:key Multibase Resolution]
        C3[Ed25519Signature2020 Proofs]
        C4[URDNA2015 JSON-LD Canonicalization]
    end

    subgraph Audit["Audit & Observability"]
        D1[security_audit_events]
        D2[trust_audit_trails]
        D3[telemetry_logs]
        D4[OpenTelemetry Tracing]
    end

    AuthN --> AuthZ --> Crypto --> Audit
```

Figure 12 — Security Architecture & Defense-in-Depth (TrustChain-AI Specification)